



7.5 物理设计

物理设计和物理模型

- 物理设计

- 在用户确认的逻辑模型基础上，以数据库系统运行效率，业务操作效率，前端应用效率等因素为出发点模型进行的调整。
- 面向物理实施过程的具体细节。
- 最终目的是转化为目标数据库的可部署的定义语言(DDL)。
- 工作内容，包括但不限于
 - 实体非正则化处理；
 - 表和字段的物理命名；
 - 确定字段的类型，长度，精度，大小写敏感等属性；
 - 增加逻辑模型中不存在的物理对象：索引，约束，分区等。

相同的事物不同的名称

- 同一个概念在不同阶段的名称

操作型文件系统	关系型理论	逻辑模型	物理模型
文件(File)	关系(Relation)	实体(Entity)	表(Table)
行记录(Record)	元组(Tuple)	实例(Instance)	行(Row)
列记录(Field)	属性(Attribute)	属性(Attribute)	字段(Column)

逻辑模型和物理模型对比

- LDM vs. PDM

	逻辑模型	物理模型
包含内容	实体、属性	表，字段
键值	主键	索引，唯一性约束
名称定义	业务名称	物理命名（受到数据库产品限制）
正则化	符合3NF	依据性能进行非正则化处理
冗余数据	无冗余数据	含冗余数据
派生数据	无派生数据	含派生数据
面向用户	业务人员和建模人员	DBA和应用开发人员

物理模型反范式处理

- 从性能和应用需求出发
 - 物理模型是以性能为出发点，支持应用需求，兼顾数据库物理限制。
 - CPU无限快，内存无限多，存储无限大，带宽无限宽，还有必要反范式处理么？
- 有限的资源，有限的硬件条件提出了物理模型反范式化的需求。
- 反范式处理需要适度进行
 - 对于特定配置的硬件系统，在满足应用功能目标和性能指标的前提下，适度进行。
 - 带来数据冗余问题。
 - 有可能导致数据不一致问题。

反范式示例 (1)

- 增加冗余列，避免频繁发生表关联操作(Join).

订单表

订单编号	订单日期	客户编号
PK		FK
1000	2010-08-01	1230008
2000	2010-11-15	1290004
3000	2010-09-30	1280003

客户表

客户编号	客户名称
PK	
1230008	王先生
1290004	李先生
1280003	赵女士

0,n



订单客户表

订单编号	订单日期	客户编号	客户名称
PK			
1000	2010-08-01	1230008	王先生
2000	2010-11-15	1290004	李先生
3000	2010-09-30	1280003	赵女士

反范式示例 (2)

- 增加冗余列，利用repeating group来减少SQL的复杂度。
 - 在前端报表应用中为了便于报表展现而采用的纵横转换。

部门销售月报

部门编号	销售月份	销售额(元)
PK	PK	
1000	2019-01	1,000,000
1000	2019-02	1,500,000
1000	2019-03	1,800,000
2000	2019-01	900,000
2000	2019-02	1,300,000
2000	2019-03	2,000,000



客户表

部门编号	一月销售额(元)	二月销售额(元)	三月销售额(元)	一季度月均销售额(元)
PK				
1000	1,000,000	1,400,000	1,800,000	1,400,000
2000	900,000	1,300,000	2,000,000	1,400,000

反范式示例 (3)

- 增加派生列，减少函数计算。

客户编号	客户名称	客户年龄
PK		
1230008	王先生	65
1290004	李先生	50
1280003	赵女士	45
1280009	张女士	20



客户编号	客户名称	客户年龄	客户分组
PK			
1230008	王先生	65	老年组
1290004	李先生	50	中年组
1280003	赵女士	45	中年组
1280009	张女士	20	青年组

反范式常见手段

- 常见反范式化处理方式
 - 增加重复组(repeating groups)
 - 预关联(pre-joins)
 - 派生字段
 - 建立汇总表或临时表
 - 表拆分（水平拆分或者垂直拆分）
- 影响
 - 并非对所有处理过程都能带来性能提升，有些负面影响需要综合考虑进行平衡。
 - 反范式会降低数据模型的灵活性。
 - 带来数据不一致的风险。

维护数据完整性

- 反范式处理后增加了数据冗余性，需要一定的管理措施来维护数据完整性。
 - 批处理维护
 - 批处理维护是指对复制列或派生列的修改积累一定的时间后，运行一批处理作业或存储过程对复制或派生列进行修改，这只能在对实时性要求不高的情况下使用。
 - 应用逻辑
 - 在应用实现过程中，在同一事务中对所有涉及的表进行增、删、改操作。
 - 风险较大，容易遗漏，特别是在需求变化时，不易于维护。
 - 触发器
 - 实时处理性高。
 - 但对于数据库压力较大，尤其是高并发环境，触发器数量需要严格控制。

建立物理化命名规范

- 建立命名规范，对实体进行物理化命名
 - 根据数据库物理特性进行命名；
 - 名称有效字符的范围（避免使用非法字符出现在名称中）；
 - 避免使用物理数据库的保留关键字；
 - 命名尽量采用富有意义、易于记忆、描述性强、简短及具有唯一性的英文词汇，不准采用汉语拼音。
 - 制定项目组范围内统一的命名规则，并严格遵守。
 - 名称缩写要达成约定。

对象命名规范示例

对象	前缀	范例	说明
表	t_	t_tablename	t_表名
普通视图	v_	v_viewname	v_视图名
索引	ix_	ix_tablename_columnname	这是最常用的索引，用ix_表示。 如果表名或字段名过长，则用表名和字段名的缩写表示，尽量使用通用缩写或去元音的缩写方式。
触发器	trg_	trg_triggername	trg_触发器名
存储过程	p_	p_procedurename	p_存储过程名
函数	f_	f_functionname	f_函数名

表的物理化

- 进行反范式化操作。
- 决定是否要分区。
 - 对于大表进行分区，减少IO扫描量，加速范围查询。
- 决定是否要拆分历史表和当前表。
 - 历史表是冷数据，可以放在低速存储上；当前表是热数据，使用高速存储。
 - 历史表可以使用压缩方法减少占用的存储空间。

字段的物理化 (1)

- 选择合适的类型，考虑以下因素：
 - 尽量使用短字段的数据类型。
 - 长度较短的数据类型不仅可以减小数据文件的大小，提升IO性能；同时也可以减小相关计算时的内存消耗，提升计算性能。
 - 比如对于整型数据，如果可以用smallint就尽量不用int，如果可以用int就尽量不用bigint。
 - 使用一致的数据类型。
 - 表关联列尽量使用相同的数据类型。如果表关联列数据类型不同，数据库必须动态地转化为相同的数据类型进行比较，这种转换会带来一定的性能消耗。
 - 选择高效数据类型。

字段的物理化 (2)

- 字段的约束
 - DEFAULT
 - 如果能够从业务层面补全字段值，就不建议使用DEFAULT约束，避免数据加载时产生不符合预期的结果。
 - NOT NULL
 - 给明确不存在NULL值的字段加上NOT NULL约束。
 - 唯一约束/主键约束
 - 主键 = 唯一 + NOT NULL。
 - 如果条件允许，就增加。
 - 检查约束
 - 检查约束因为对于数据质量提出了要求，不满足约束的数据在插入数据表会导致SQL失败。

索引的创建和使用

- 可以增加索引的情况：
 - 在经常需要搜索查询的列；
 - 在作为主键的列上创建索引，强制该列的唯一性；
 - 在经常使用连接的列上创建索引；
 - 在经常需要根据范围进行搜索的列上创建索引；
 - 在经常需要排序的列上创建索引；
 - 在经常使用WHERE子句的列上创建索引。
- 索引建多了，会有负面影响
 - 占用更多的空间；
 - 插入基表数据的效率会下降。
- 删除无效的索引，避免空间浪费

其他物理化手段

- 根据其他特定需求：
 - 是否采用压缩。
 - 是否需要对数据进行加密。
 - 是否需要对数据进行脱敏。

使用建模软件

- 使用建模软件来进行逻辑建模和物理建模：
 - 功能强大而丰富；
 - 正向生成DDL，反向解析；
 - 在逻辑模型和物理模型中自由切换使用视图；
 - 全面满足建模中的各种需求，高效进行建模。
- 相关软件：
 - CA ERWin；
 - SAP PowerDesigner；
 - ER/Studio；
 - pgModeler；
 - Dbeaver Community；
 - ...

物理模型产出物

- 物理数据模型；
- 物理模型命名规范；
- 物理数据模型设计说明书；
- 生成DDL建表语句。